

BÁO CÁO PROJECT

Xây dựng CPU Von Neumann Pipeline 5 tầng kết hợp Unified Cache

Thiết kế và mô phỏng CPU 16-bit kiến trúc Von Neumann Pipeline 5 tầng kết hợp Unified Direct-Mapped Cache

Tên đề tài: Thiết kế và mô phỏng CPU 16-bit kiến trúc Von Neumann Pipeline 5 tầng kết hợp Unified Direct-Mapped Cache

Nhóm thực hiện: Ghi đầy đủ tên 4 thành viên

Môn học: Kiến trúc máy tính / Thiết kế hệ thống số / Verilog HDL

Giảng viên hướng dẫn:

Thời gian:

May 16, 2026

Tóm tắt đóng góp của các thành viên

Báo cáo được định dạng theo tinh thần paper chuẩn APA với trang bìa, căn lề 1 inch, đánh số trang và bố cục nội dung học thuật; phần trích dẫn và tài liệu tham khảo sử dụng kiểu đánh số tương tự IEEE numerical citation style [1, 2]. Mức độ đóng góp dưới đây được sắp xếp từ cao đến thấp, dựa trên vai trò tích hợp hệ thống, khối lượng thiết kế, mức độ ảnh hưởng đến kiến trúc cuối cùng và phần kiểm chứng.

1. **Nguyễn Nhật Phong — Control Unit & System Integrator.** Phụ trách chính việc xác định kiến trúc CPU 16-bit, tổ chức pipeline 5 tầng, thiết kế ISA, control unit, hazard detection, forwarding, stall và flush. Thành viên này cũng đảm nhiệm vai trò tích hợp các khối datapath, memory subsystem và testbench thành hệ thống hoàn chỉnh, đồng thời tổng hợp các phần giới thiệu, yêu cầu hệ thống, kiến trúc tổng thể, đánh giá và kết luận.
2. **Nguyễn Vũ Hoàng — Memory & Cache Specialist.** Phụ trách thiết kế unified direct-mapped cache, main memory có delay và memory arbiter để xử lý tranh chấp truy cập giữa IF stage và MEM stage. Đóng góp chính gồm mô tả cache hit/miss, read miss refill, write-through, no-write-allocate, FSM cache và kiểm thử các trường hợp read/write với RAM có độ trễ.
3. **Vũ Xuân Hải — Datapath & ALU Designer.** Phụ trách thiết kế datapath, ALU, register file, immediate generator và các MUX chọn toán hạng. Đóng góp tập trung vào việc mô tả đường đi dữ liệu qua các stage, kiểm soát thanh ghi R0, các phép toán số học/logic và kiểm thử ALU/register file.
4. **Lê Hoàng Dũng — Verification & Tooling Engineer.** Phụ trách assembler Python, demo program, testbench, waveform và bảng pass/fail. Đóng góp chính gồm chuyển assembly sang file .mem, xây dựng kịch bản kiểm thử module/tích hợp, mô tả kết quả simulation và chuẩn bị phụ lục về cấu trúc thư mục, lệnh mô phỏng và kế hoạch làm việc.

Tổng thể, nhóm phân công theo hướng mỗi thành viên chịu trách nhiệm một lớp của hệ thống: điều khiển và tích hợp, bộ nhớ/cache, datapath/ALU, và verification/tooling. Cách phân chia này giúp quá trình phát triển bám sát luồng thiết kế phần cứng hiện đại: xác định ISA, thiết kế module, mô phỏng từng khối, tích hợp pipeline, kiểm thử hazard/cache và đánh giá kết quả.

Contents

Tóm tắt đóng góp của các thành viên	1
1 Giới thiệu đề tài	5
1.1 Bối cảnh đề tài	5
1.2 Tổng quan đề tài	6
1.3 Mục tiêu đề tài	7
1.4 Thông tin cần kiểm chứng và nguồn trích dẫn	8
1.5 Phạm vi thực hiện	9
2 Phương pháp luận	9
2.1 Cơ sở lý thuyết nền tảng	9
2.1.1 Kiến trúc Von Neumann	9
2.1.2 CPU 16-bit	10
2.1.3 Pipeline 5 tầng	10
2.1.4 Instruction Set Architecture — ISA	11
2.1.5 Cấu trúc Tổng thể Đường ống Dữ liệu (Datapath)	14
2.1.6 Khối Tính toán Trung tâm (ALU)	15
2.1.7 Tập thanh ghi (Register File)	16
2.1.8 Control Unit	17
2.1.9 Cache Memory	17
2.1.10 Main Memory có Delay	18
2.1.11 Hazard trong Pipeline	18
2.2 Phân tích yêu cầu hệ thống	19
2.2.1 Yêu cầu chức năng	19

2.2.2	Yêu cầu phi chức năng	19
2.3	Thiết kế hệ thống	19
2.3.1	Tổng quan kiến trúc hệ thống	19
2.3.2	Luồng pipeline và xử lý xung đột	20
2.3.3	Tổng quan giao diện module	21
2.3.4	Ghi chú phạm vi thiết kế	21
2.3.5	Kiến trúc tổng thể	22
2.3.6	Thiết kế module CPU Core	22
2.3.7	Thiết kế Datapath & ALU	23
2.3.8	Thiết kế Control Unit	23
2.3.9	Thiết kế Pipeline Registers	23
2.3.10	Thiết kế Hazard Detection & Forwarding	23
2.3.11	Thiết kế Memory Arbiter	24
2.3.12	Thiết kế Direct-Mapped Cache	24
2.3.13	Thiết kế Main Memory	25
2.3.14	Thiết kế Python Assembler	25
2.4	Quá trình xây dựng hệ thống	27
2.4.1	Quy trình triển khai	27
2.4.2	Xây dựng Datapath, ALU, Register File	27
2.4.3	Xây dựng Cache, RAM và Memory Arbiter	27
2.4.4	Xây dựng Control, Pipeline và CPU Core	28
2.4.5	Xây dựng Testbench, Assembler và Demo Program	29
3	Kết quả và bàn luận	30
3.1	Chiến lược kiểm thử	30

3.2	Testbench từng module	30
3.3	Kiểm thử chương trình Assembly	30
3.4	Waveform minh họa	32
3.5	Bảng tổng hợp kết quả kiểm thử	32
3.6	Đánh giá hệ thống	33
3.6.1	Kết quả đạt được	33
3.6.2	Hạn chế	33
3.6.3	Hướng phát triển	34
3.7	Kết luận	34
4	Tài liệu tham khảo	34
5	Phụ lục	35
5.1	Cấu trúc thư mục project	35
5.2	Lệnh mô phỏng	35
5.3	Code quan trọng	36
5.4	Gợi ý chia file trong thư mục docs/	36
5.5	Checklist hoàn thành báo cáo	37
5.6	Lưu ý khi viết báo cáo	39

1. Giới thiệu đề tài

Người viết chính: Lê Hoàng Dũng

1.1. Bối cảnh đề tài

Bộ xử lý trung tâm (CPU) đóng vai trò cốt lõi trong việc thực thi chương trình và điều phối luồng dữ liệu của hệ thống máy tính. Trong kiến trúc máy tính hiện đại, việc mô hình hóa CPU thông qua mối tương quan giữa tập lệnh, đường dẫn dữ liệu (datapath), khối điều khiển (control unit), bộ nhớ và cơ chế vào/ra là một phương pháp tiếp cận nền tảng [3, 4]. Dựa trên cơ sở lý thuyết này, đề tài đề xuất thiết kế một vi kiến trúc CPU 16-bit theo hướng tối giản. Dù được tinh gọn, thiết kế vẫn đảm bảo tính toàn vẹn trong việc mô phỏng các chu trình cốt lõi của vi kiến trúc: nạp lệnh, giải mã lệnh, thực thi ALU, truy cập bộ nhớ, ghi trả kết quả, cũng như các cơ chế xử lý xung đột (hazard) trên đường ống.

Kiến trúc Von Neumann được áp dụng làm nền tảng phát triển dựa trên đặc trưng sử dụng chung không gian bộ nhớ cho cả lệnh (instruction) và dữ liệu (data) [4, 5]. Mô hình chương trình được lưu trữ (stored-program) này cho phép minh họa một cách trực quan mối quan hệ phụ thuộc giữa lõi CPU và hệ thống con bộ nhớ (memory subsystem). Tuy nhiên, hạn chế cố hữu của kiến trúc Von Neumann là nguy cơ phát sinh xung đột cấu trúc (*structural hazard*) khi các tầng nạp lệnh (IF) và truy cập bộ nhớ (MEM) đồng thời yêu cầu tài nguyên. Để giải quyết vấn đề này, hệ thống được tích hợp thêm một bộ điều phối bộ nhớ (*memory_arbiter*) nhằm quản lý và phân giải các yêu cầu truy xuất một cách hiệu quả.

Nhằm tối ưu hóa thông lượng (throughput) của hệ thống, kỹ thuật đường ống (pipeline) 5 tầng bao gồm IF (Nạp lệnh) – ID (Giải mã) – EX (Thực thi) – MEM (Truy cập bộ nhớ) – WB (Ghi trả kết quả) được triển khai. Đây là mô hình kinh điển giúp quan sát và xử lý triệt để các dạng xung đột dữ liệu (data hazard), xung đột tải - sử dụng (load-use hazard) và xung đột điều khiển (control hazard) [3, 4]. Hơn thế nữa, để khắc phục nút thắt cổ chai về hiệu suất giữa tốc độ xử lý của CPU và độ trễ của bộ nhớ chính, một hệ thống bộ nhớ đệm (cache) đã được thiết kế bổ sung. Cấu trúc cache ánh xạ trực tiếp (direct-mapped), tuy đơn giản, nhưng vẫn đáp ứng đầy đủ yêu cầu kiểm chứng các tham số vi kiến trúc quan trọng như thẻ (tag), chỉ số (index), bit hợp lệ (valid bit), các trạng thái trúng/trượt (hit/miss), quá trình nạp lại (refill) và chính sách ghi (write policy) [3, 4].

Về phương pháp thực thi, ngôn ngữ mô tả phần cứng Verilog HDL (được chuẩn hóa theo tiêu

chuẩn IEEE 1364 [6]) được lựa chọn để hiện thực hóa thiết kế ở mức thanh ghi chuyển (RTL - Register Transfer Level). Khả năng biểu diễn linh hoạt các module, tín hiệu, mạch tổ hợp và mạch tuần tự của Verilog cho phép chuyển đổi chính xác các nền tảng lý thuyết về vi kiến trúc thành các thực thể phần cứng vật lý. Đồng thời, việc sử dụng Verilog tạo điều kiện thuận lợi cho quá trình định nghĩa giao diện (interface) tường minh và xây dựng môi trường kiểm chứng (testbench) toàn diện, đảm bảo độ tin cậy của thiết kế hệ thống.

1.2. Tổng quan đề tài

Mục tiêu trọng tâm của nghiên cứu là thiết kế và mô phỏng một vi xử lý (CPU) 16-bit dựa trên kiến trúc Von Neumann. Hệ thống tích hợp cơ chế đường ống lệnh (pipeline) 5 tầng, kết hợp với bộ nhớ đệm ánh xạ trực tiếp thống nhất (unified direct-mapped cache) và bộ nhớ chính (main memory) có mô phỏng độ trễ truy xuất. Kiến trúc mức cao nhất (top-level) của hệ thống được tổ chức theo luồng điều khiển và dữ liệu như sau:

`cpu_core` $\rightarrow$ `memory_arbiter` $\rightarrow$ `direct_mapped_cache` $\rightarrow$ `main_memory`

Trong cấu trúc này, module `cpu_core` đảm nhiệm vai trò thực thi tập lệnh thông qua cơ chế pipeline. Module `memory_arbiter` có chức năng phân giải và hợp nhất các yêu cầu truy xuất từ phân đoạn Nạp lệnh (IF) và Truy xuất bộ nhớ (MEM) thành một cổng giao tiếp vật lý duy nhất nối với bộ nhớ đệm. Tiếp đó, `direct_mapped_cache` quản lý các trạng thái trúng/trượt (hit/miss) và thực thi chính sách ghi. Cuối cùng, `main_memory` đóng vai trò mô phỏng bộ nhớ RAM đánh địa chỉ theo từ (word-addressed) với các thông số độ trễ được cấu hình sẵn.

Table 1: Tổng quan các thành phần chính của hệ thống

Thành phần	Vai trò	Nội dung cần kiểm chứng
<code>cpu_core</code>	Thực thi tập lệnh (ISA) 16-bit qua pipeline 5 tầng	Cập nhật PC, giải mã lệnh, tính toán ALU, truy cập bộ nhớ, ghi trả kết quả, và xử lý trạng thái HALT
<code>register_file</code>	Quản lý tập 8 thanh ghi 16-bit	Tính chính xác khi đọc/ghi địa chỉ; duy trì giá trị R0 luôn bằng 0
<code>alu</code>	Xử lý các phép toán số học, logic, dịch bit (shift) và so sánh	Độ chính xác của kết quả 16-bit và các cờ trạng thái (zero , negative , overflow)

(tiếp trang sau)

Table 1: Tổng quan các thành phần chính của hệ thống (tiếp)

Thành phần	Vai trò	Nội dung cần kiểm chứng
<code>hazard_detection_unit</code>	Phát hiện và phân giải các tình huống cần đình trệ (stall) hoặc xả (flush) pipeline	Xung đột tải-sử dụng (load-use hazard), đình trệ do trượt cache, và cơ chế chèn chu kỳ trống (bubble)
<code>forwarding_unit</code>	Lựa chọn và chuyển tiếp (forward) dữ liệu về phân đoạn EX	Chuyển tiếp kết quả từ các chốt EX/MEM hoặc MEM/WB; ngăn chặn chuyển tiếp sai vào R0
<code>memory_arbiter</code>	Điều phối các yêu cầu truy xuất từ tầng IF và MEM đến cache	Mức độ ưu tiên của MEM so với IF; kích hoạt trạng thái stall cho IF khi xảy ra tranh chấp tài nguyên
<code>direct_mapped_cache</code>	Bộ nhớ đệm cấp 1 (L1) dùng chung cho lệnh và dữ liệu	Các cơ chế trùng/trượt, nạp lại (refill), chính sách ghi xuyên (write-through) và không cấp phát khi ghi (no-write-allocate)
<code>main_memory</code>	Mô phỏng bộ nhớ RAM có độ trễ truy xuất	Tín hiệu ready chỉ bật khi hoàn tất yêu cầu; tính toàn vẹn của dữ liệu trong quá trình đọc/ghi
<code>assembler.py</code>	Trình biên dịch mã hợp ngữ (assembly) sang tập tin <code>.mem</code>	Logic mã hóa (encoding) các tập lệnh R-type, I-type, J-type; xử lý chính xác nhãn (label) và hằng số (immediate)

1.3. Mục tiêu đề tài

Mục tiêu tổng quát của project là xây dựng một mô hình CPU có thể mô phỏng được toàn bộ luồng thực thi instruction từ chương trình assembly đến kết quả trên register/memory. Các mục tiêu cụ thể bao gồm:

- Thiết kế CPU 16-bit với instruction width, data width và address width đều là 16-bit.
- Xây dựng tập 8 thanh ghi tổng quát từ R0 đến R7, trong đó R0 = 0.
- Định nghĩa ISA gồm các nhóm lệnh R-type, I-type và J-type, đủ để kiểm thử arithmetic, memory, branch, jump và halt.
- Triển khai pipeline 5 tầng gồm IF, ID, EX, MEM và WB.
- Thiết kế control unit, pipeline registers, hazard detection, forwarding, stall và flush.

- Sử dụng unified direct-mapped cache để mô phỏng hệ thống bộ nhớ chung instruction/-data.
- Mô phỏng main memory có delay từ 5 đến 10 chu kỳ để kiểm tra stall khi bộ nhớ chưa sẵn sàng.
- Xây dựng Python assembler để dịch file `.asm` sang file `.mem` dùng cho `$readmemh`.
- Xây dựng testbench cho từng module và testbench tích hợp toàn hệ thống.
- Tổng hợp kết quả kiểm thử, waveform và bảng pass/fail để đánh giá mức độ hoàn thành.

1.4. Thông tin cần kiểm chứng và nguồn trích dẫn

Báo cáo phân biệt giữa hai loại thông tin: thông tin nền tảng cần trích dẫn từ tài liệu học thuật/chuẩn kỹ thuật, và thông tin implementation cần kiểm chứng bằng simulation của chính project. Cách phân biệt này giúp phần viết tránh nhầm lẫn giữa kiến thức đã được công bố và kết quả do nhóm tự triển khai.

Table 2: Các thông tin cần kiểm chứng trong báo cáo

Nhóm thông tin	Nội dung cần kiểm chứng	Nguồn/Phương pháp
Nền tảng Von Neumann	Stored-program model, instruction và data dùng chung bộ nhớ	Tài liệu EDVAC và giáo trình tổ chức máy tính [4, 5]
Pipeline	Ý nghĩa pipeline, stage IF/ID/EX/MEM/WB, hazard và cơ chế stall/flush/forwarding	Giáo trình kiến trúc/tổ chức máy tính [3, 4]
Cache	Locality, cache hit/miss, direct-mapped cache, write-through và memory hierarchy	Giáo trình kiến trúc máy tính [3, 4]
Verilog HDL	Vai trò Verilog trong mô tả, mô phỏng và kiểm chứng phần cứng số	Chuẩn IEEE 1364 [6]
ISA của nhóm	Opcode, funct, định dạng R/I/J, quy ước branch/jump/HALT	Bảng ISA trong báo cáo và test assembler
Datapath/Control	Đường đi dữ liệu, control signal, pipeline register, forwarding và hazard detection	Testbench module và waveform tích hợp
Memory subsystem	Arbiter ưu tiên MEM, cache hit/miss, RAM delay và tín hiệu <code>ready</code>	Testbench cache/RAM/arbiter
Tooling	<code>assembler.py</code> mã hóa đúng file <code>.asm</code> sang <code>.mem</code>	So sánh output hex và chạy simulation
Kết quả project	Pass/fail, register/memory cuối chương trình, waveform minh họa	Log mô phỏng, waveform và bảng kết quả

1.5. Phạm vi thực hiện

Project tập trung vào mô phỏng logic CPU bằng Verilog HDL ở mức RTL, chưa đặt mục tiêu tổng hợp lên FPGA hoặc tối ưu timing vật lý. Cache sử dụng cấu trúc direct-mapped với line size 1 word. Chính sách ghi sử dụng *write-through*; write miss policy là *no-write-allocate*. Branch được resolve ở EX stage, vì vậy khi branch hoặc jump đổi luồng điều khiển, pipeline cần flush các instruction sai đường đi. Các kỹ thuật nâng cao như superscalar, out-of-order execution, branch prediction, cache nhiều cấp, write-back cache và non-blocking cache được xem là ngoài phạm vi của phiên bản này.

2. Phương pháp luận

Phần này mô tả cấu trúc, chức năng chính của sản phẩm, ngôn ngữ mô tả phần cứng Verilog HDL và các công cụ được chọn để triển khai, mô phỏng, kiểm thử hệ thống CPU pipeline kết hợp unified cache.

2.1. Cơ sở lý thuyết nền tảng

2.1.1. Kiến trúc Von Neumann

Người viết: Lê Hoàng Dũng.

Kiến trúc Von Neumann là mô hình máy tính stored-program, trong đó lệnh và dữ liệu được lưu trong cùng một bộ nhớ chính và được truy cập thông qua hệ thống bus chung [4, 5]. Một hệ thống cơ bản gồm CPU, bộ nhớ chính và hệ thống vào/ra; bên trong CPU thường có ALU để thực hiện phép toán, control unit để giải mã và điều phối lệnh, cùng các thanh ghi như PC và IR để lưu trạng thái tạm thời của quá trình thực thi.

Chu trình hoạt động điển hình là *fetch-decode-execute*: CPU lấy lệnh từ địa chỉ do PC trỏ tới, giải mã lệnh trong control unit, sau đó thực thi bằng ALU, register file hoặc bộ nhớ tùy loại lệnh [4]. Cách tổ chức này đơn giản và phù hợp cho mục tiêu mô phỏng học thuật vì toàn bộ instruction/data có thể được quan sát qua một đường bộ nhớ thống nhất.

Hạn chế chính của mô hình này là *Von Neumann bottleneck*: instruction và data chia sẻ cùng bộ nhớ/bus nên CPU có thể phải chờ khi vừa cần fetch instruction vừa cần truy cập dữ liệu [3]. Trong project này, đặc điểm đó được mô phỏng bằng unified cache; IF stage và MEM stage

cùng gửi request tới cache, còn `memory_arbiter` điều phối truy cập và ưu tiên MEM stage khi có tranh chấp.

2.1.2. CPU 16-bit

Người viết: Nguyễn Nhật Phong và Vũ Xuân Hải.

CPU được thiết kế với độ rộng dữ liệu, địa chỉ và instruction đều là 16-bit. Hệ thống gồm 8 thanh ghi tổng quát, trong đó thanh ghi R0 luôn bằng 0. CPU hỗ trợ các nhóm lệnh cơ bản như sau:

- R-type: ADD, SUB, AND, OR, SLT.
- I-type: ADDI, LW, SW, BEQ, BNE.
- J-type: J, HALT.

Table 3: Cấu hình tổng quan CPU

Thành phần	Cấu hình
Data width	16-bit
Address width	16-bit
Instruction width	16-bit
Register file	8 x 16-bit
Pipeline	5 tầng
Cache	Unified direct-mapped
Main memory	Có delay

2.1.3. Pipeline 5 tầng

Người viết: Nguyễn Nhật Phong.

Pipeline gồm 5 stage: IF, ID, EX, MEM và WB. Cơ chế này cho phép nhiều lệnh được xử lý chồng lấn, từ đó tăng throughput của hệ thống. Tuy nhiên, việc pipeline hóa cũng làm phát sinh các loại hazard, đòi hỏi phải có cơ chế stall, forwarding và flush.

Table 4: Các stage trong pipeline

Stage	Tên	Vai trò
IF	Instruction Fetch	Lấy lệnh từ bộ nhớ/cache
ID	Instruction Decode	Giải mã lệnh, đọc thanh ghi
EX	Execute	Thực hiện ALU, tính địa chỉ, xét branch
MEM	Memory Access	Load/store dữ liệu
WB	Write Back	Ghi kết quả về register file

IF $\rightarrow$ ID $\rightarrow$ EX $\rightarrow$ MEM $\rightarrow$ WB

2.1.4. Instruction Set Architecture — ISA

Người viết: Lê Hoàng Dũng.

ISA là bản đặc tả chính thức cho CPU 16-bit của nhóm bao gồm các quy tắc chung, sau đó là định dạng lệnh, bảng mã lệnh, chi tiết ALU, quy ước assembly.

Table 5: Các quyết định chính của ISA

Mục	Đặc tả
Độ rộng lệnh	16 bit
Độ rộng dữ liệu	16 bit
Thanh ghi	8 thanh ghi tổng quát: R0 đến R7
Luật R0	Đọc luôn trả 16'h0000; mọi lệnh ghi vào R0 bị bỏ qua
Bộ nhớ	Word-addressed, mỗi word rộng 16 bit
PC increment	Mỗi lệnh tuần tự dùng $PC = PC + 1$
NOP	16'h0000, tương đương ADD R0, R0, R0

Định dạng lệnh ISA dùng ba định dạng lệnh: R-type cho các phép toán thanh ghi, I-type cho immediate/load/store/branch và J-type cho jump tuyệt đối.

Table 6: Định dạng bit của các loại lệnh

Format	Trường bit	Ý nghĩa
R-type	[15:12] opcode	Opcode, luôn là 4'h0 với nhóm R-type
R-type	[11:9] rs	Toán hạng nguồn 1
R-type	[8:6] rt	Toán hạng nguồn 2
R-type	[5:3] rd	Thanh ghi đích
R-type	[2:0] funct	Chọn phép tính ALU

Continued on next page

Table 6: Định dạng bit của các loại lệnh (Continued)

Format	Trường bit	Ý nghĩa
I-type	[15:12] opcode	Opcode của lệnh I-type
I-type	[11:9] rs	Toán hạng nguồn hoặc base register cho LW/SW
I-type	[8:6] rt	Đích với ADDI/LW, dữ liệu với SW, toán hạng so sánh với BEQ/BNE
I-type	[5:0] imm6	Immediate 6 bit, được sign-extend trước khi vào ALU
J-type	[15:12] opcode	Opcode của lệnh nhảy hoặc HALT
J-type	[11:0] address	Địa chỉ nhảy tuyệt đối 12 bit theo word address

Opcode
Table 7: Bảng opcode

Opcode	Mnemonic	Loại
4'h0	R-type	R
4'h1	ADDI	I
4'h2	LW	I
4'h3	SW	I
4'h4	BEQ	I
4'h5	BNE	I
4'h6	J	J
4'hF	HALT	J-like

Funct và mã ALU Với R-type, opcode luôn là 4'h0; phép toán cụ thể được chọn bằng field `funct[2:0]`. Control unit zero-extend field này thành `alu_op = {1'b0, funct}`, nên mã ALU hợp lệ nằm từ 4'b0000 đến 4'b0111.

Table 8: Bảng funct của R-type

Funct	alu_op	Lệnh	Mô tả ngắn
3'h0	4'b0000	ADD	$rd = rs + rt$, có kiểm tra overflow có dấu
3'h1	4'b0001	SUB	$rd = rs - rt$, có kiểm tra overflow có dấu
3'h2	4'b0010	AND	AND từng bit, <code>overflow = 0</code>
3'h3	4'b0011	OR	OR từng bit, <code>overflow = 0</code>
3'h4	4'b0100	XOR	XOR từng bit, <code>overflow = 0</code>
3'h5	4'b0101	SLT	So sánh có dấu: $rd = (rs < rt) ? 1 : 0$
3'h6	4'b0110	SLL	Dịch trái logic: $rd = rs \ll rt[3:0]$
3'h7	4'b0111	SRL	Dịch phải logic: $rd = rs \gg rt[3:0]$

Table 9: Chi tiết hoạt động ALU của ALU

Lệnh	Kết quả	Cờ trạng thái
ADD	$result = rs + rt$, kết quả wrap 16 bit	$overflow = 1$ khi cộng hai số cùng dấu nhưng kết quả khác dấu; $zero=(result==0)$; $negative=result[15]$
SUB	$result = rs - rt$, kết quả wrap 16 bit	$overflow = 1$ khi trừ hai số khác dấu và kết quả sai dấu; $zero=(result==0)$; $negative=result[15]$
AND	$result = rs \& rt$	$overflow = 0$; cập nhật $zero, negative$ theo kết quả
OR	$result = rs rt$	$overflow = 0$; cập nhật $zero, negative$ theo kết quả
XOR	$result = rs \wedge rt$	$overflow = 0$; cập nhật $zero, negative$ theo kết quả
SLT	$(\$signed(rs) < \$signed(rt)) ? 0001 : 0000$	So sánh có dấu; $overflow=0$; $negative=0$; $zero=1$ khi $rs \geq rt$
SLL	$result = rs \ll rt[3:0]$	Dùng 4 bit thấp của rt làm shift amount (0–15); $overflow=0$
SRL	$result = rs \gg rt[3:0]$	Dịch phải logic, chèn 0 bên trái, không giữ bit dấu; $overflow=0$

Ví dụ: nếu $rs = 0x0001$ và $rt = 0x0003$, lệnh SLL rd, rs, rt cho $result = 0x0008$. Nếu $rs = 0x8000$ và $rt = 0x0001$, lệnh SRL rd, rs, rt cho $result = 0x4000$. Với SLT, nếu $R3 = 0xFFFF$ (-1) và $R2 = 0x0001$ (+1) thì SLT $R1, R3, R2$ cho $R1 = 1$.

Quy ước assembly

Table 10: Cú pháp assembly khuyến nghị

Cú pháp	Ý nghĩa
ADD rd, rs, rt	$rd = rs + rt$
SUB rd, rs, rt	$rd = rs - rt$
AND/OR/XOR rd, rs, rt	Phép logic tương ứng giữa rs và rt
SLT rd, rs, rt	So sánh có dấu, ghi 1 nếu $rs < rt$, ngược lại ghi 0
SLL rd, rs, rt	$rd = rs \ll rt[3:0]$
SRL rd, rs, rt	$rd = rs \gg rt[3:0]$
ADDI $rt, rs, imm6$	$rt = rs + sign_extend(imm6)$
LW $rt, imm6(rs)$	$rt = MEM[rs + sign_extend(imm6)]$
SW $rt, imm6(rs)$	$MEM[rs + sign_extend(imm6)] = rt$
BEQ $rs, rt, label$	Nếu $rs == rt$, PC nhận địa chỉ nhánh
BNE $rs, rt, label$	Nếu $rs != rt$, PC nhận địa chỉ nhánh
J $label$	PC nhận địa chỉ tuyệt đối 12 bit
HALT	Dừng pipeline, bật $halt = 1$

Branch, jump và halt Branch được resolve ở EX stage. Địa chỉ đích branch được tính bằng:

```
branch_target = PC_plus_1 + sign_extend(imm6)
imm6 = label_address - (branch_pc + 1)
```

Hai lệnh BEQ và BNE dùng phép trừ trong ALU để kiểm tra điều kiện; nếu `zero = 1`, hai thành ghi bằng nhau.

Jump dùng địa chỉ tuyệt đối 12 bit theo word address:

```
jump_target = zero_extend(address[11:0])
range       = 0x000 .. 0xFFF
```

Khi `opcode = 4'hF`, lệnh HALT bật `halt = 1`, pipeline không fetch lệnh mới và PC giữ nguyên. Trong thiết kế hiện tại, HALT được implement ở tầng ID và tín hiệu `halt` được truyền qua pipeline register.

Cờ trạng thái ALU
Table 11: Cờ trạng thái ALU

Cờ	Kiểu/bit	Mô tả
zero	wire	Bằng 1 khi <code>result == 16'h0000</code> ; dùng cho BEQ/BNE
negative	result[15]	Bằng 1 khi kết quả là số âm theo two's complement
overflow	reg	Bằng 1 khi ADD/SUB tràn số có dấu; bằng 0 với logic/shift

Trong implementation, **zero** là tín hiệu **assign** dạng combinational, còn **negative** và **overflow** là kiểu **reg** được cập nhật trong khối **always @(*)**.

2.1.5. Cấu trúc Tổng thể Đường ống Dữ liệu (Datapath)

Thực hiện: Vũ Xuân Hải

Đường ống dữ liệu (Datapath) là hệ thống mạch kết nối lõi, chịu trách nhiệm định tuyến, lưu trữ và biến đổi dữ liệu xuyên suốt quá trình thực thi lệnh của vi xử lý. Trong kiến trúc 5 tầng (5-Stage Pipeline) kết hợp với nền tảng bộ nhớ Von Neumann, Datapath được cấu thành từ các module tổ hợp lõi (Register File, ALU, Immediate Generator, Multiplexers) và được phân tách đồng bộ bởi hệ thống thanh ghi đường ống (Pipeline Registers).

Luồng dữ liệu tổng quát vận hành qua 5 chu kỳ khép kín, định hình chuỗi hành vi của phần cứng đối với một mã lệnh 16-bit:

- **Tầng Lấy lệnh (Instruction Fetch - IF):** Con trỏ lệnh (Program Counter) cấp địa chỉ để truy xuất mã lệnh từ bộ nhớ. Mã lệnh thô sau đó được chốt vào thanh ghi cách ly IF/ID để chuyển sang chu kỳ tiếp theo.
- **Tầng Giải mã (Instruction Decode - ID):** Tập thanh ghi (Register File) thực hiện trích xuất dữ liệu của toán hạng **rs** và **rt**. Song song đó, khối Immediate Generator tiến hành nội suy và mở rộng hằng số có trong mã lệnh đạt chuẩn 16-bit. Toàn bộ dữ liệu này được đẩy vào chốt ID/EX.
- **Tầng Thực thi (Execute - EX):** Mạng lưới bộ định tuyến (MUX) tiến hành bẻ ghi nguồn dữ liệu đầu vào. Khối tính toán trung tâm (ALU) tiếp nhận toán hạng để thực thi các phép toán số học/logic, đồng thời sinh ra các cờ trạng thái (**zero**, **negative**, **overflow**). Kết quả thực thi được chốt vào ranh giới EX/MEM.
- **Tầng Truy xuất Bộ nhớ (Memory - MEM):** Tại pha này, hệ thống tương tác với bộ nhớ chính. Đối với các lệnh **LOAD/STORE**, tín hiệu đọc/ghi RAM được kích hoạt. Đối với các lệnh tính toán thuần túy, kết quả từ ALU sẽ chảy xuyên qua pha này và được bảo toàn vào chốt MEM/WB.
- **Tầng Ghi trả (Write-Back - WB):** Khối MUX cuối cùng quyết định luồng dữ liệu hợp lệ (từ ALU hoặc từ bộ nhớ) để dội ngược về Tập thanh ghi. Thao tác cập nhật dữ liệu vào thanh ghi đích **rd** được đồng bộ hóa theo sườn lên của xung nhịp, chính thức khép lại vòng đời thực thi của lệnh.

(Ghi chú bổ sung: Sơ đồ khối trực quan mô tả mạng lưới định tuyến vật lý và các cổng giao tiếp bên trong Datapath sẽ được cập nhật trong phiên bản báo cáo tổng hợp cuối cùng nhằm đối chiếu với các đặc tả kỹ thuật nêu trên.)

Thực hiện: Vũ Xuân Hải

2.1.6. Khối Tính toán Trung tâm (ALU)

Khối ALU (Arithmetic Logic Unit) được thiết kế để thực hiện các phép toán số học và logic cơ bản cho luồng dữ liệu 16-bit. Khối hoạt động dựa trên mạch tổ hợp (combinational logic) và nhận mã lệnh điều khiển từ Control Unit để quyết định phép toán cần thực thi.

ALU hỗ trợ tổng cộng 8 phép toán bao gồm:

- **Số học:** ADD (cộng), SUB (trừ).
- **Logic:** AND, OR, XOR.
- **Dịch bit:** SLL (dịch trái logic), SRL (dịch phải logic).
- **So sánh:** SLT (Set on Less Than - so sánh nhỏ hơn có dấu).

Để hỗ trợ quá trình kiểm tra điều kiện và rẽ nhánh của CPU, ALU xuất ra 3 cờ trạng thái (status flags) sau mỗi phép tính:

- **Cờ zero:** Đạt mức logic 1 khi kết quả tính toán bằng 16'h0000.
- **Cờ negative:** Đạt mức logic 1 khi kết quả mang giá trị âm (bit dấu MSB bằng 1).
- **Cờ overflow:** Đạt mức logic 1 khi xảy ra hiện tượng tràn số trong các phép toán cộng hoặc trừ có dấu.

2.1.7. Tập thanh ghi (Register File)

Tập thanh ghi nội bộ (Register File) cung cấp không gian lưu trữ tạm thời với tốc độ truy xuất cao, đóng vai trò cung cấp toán hạng đầu vào cho ALU và lưu trữ kết quả trả về.

Đặc tả kỹ thuật của tập thanh ghi bao gồm:

- **Cấu trúc không gian:** Hệ thống gồm 8 thanh ghi, mỗi thanh ghi có độ rộng 16-bit.
- **Cổng giao tiếp (Ports):** Hỗ trợ đọc xuất cùng lúc 2 toán hạng thông qua 2 cổng đọc độc lập (**rs**, **rt**) và lưu dữ liệu thông qua 1 cổng ghi (**rd**).
- **Cơ chế đồng bộ:** Quá trình đọc diễn ra bất đồng bộ (asynchronous read) để đảm bảo tính liên tục của luồng dữ liệu. Quá trình ghi được đồng bộ theo sườn lên của xung nhịp hệ thống (positive edge-triggered) khi tín hiệu cho phép ghi được kích hoạt.
- **Đặc tả thanh ghi R0:** Thanh ghi R0 (tại địa chỉ 3'b000) được gắn cứng (hardwired) với giá trị 16'h0000. Thiết kế phần cứng mặc định không cho phép ghi đè lên thanh ghi này nhằm phục vụ các phép tính cần hằng số 0.

2.1.8. Control Unit

Người viết: Nguyễn Nhật Phong.

Control unit sinh các tín hiệu điều khiển dựa trên `opcode` và `funct`. Các tín hiệu chính gồm `reg_dst`, `alu_src`, `mem_to_reg`, `reg_write`, `mem_read`, `mem_write`, `branch`, `branch_ne`, `jump`, `alu_op` và `halt`.

Table 12: Bảng tín hiệu điều khiển theo lệnh

Lệnh	reg_write	mem_read	mem_write	alu_src	mem_to_reg	branch	jump
ADD/SUB/.../SLT	1	0	0	0	0	0	0
ADDI	1	0	0	1	0	0	0
LW	1	1	0	1	1	0	0
SW	0	0	1	1	0	0	0
BEQ/BNE	0	0	0	0	0	1	0
J	0	0	0	0	0	0	1
HALT	0	0	0	0	0	0	0

2.1.9. Cache Memory

Người viết: Nguyễn Vũ Hoàng.

Cache có vai trò giảm thời gian truy cập bộ nhớ chính. Project sử dụng unified direct-mapped cache, nghĩa là instruction và data dùng chung một cache. Với cấu hình 16 lines, line size 1 word, ta có:

- Tag = `address[15:4]`
- Index = `address[3:0]`

Table 13: Chính sách cache

Chính sách	Ý nghĩa
Write-through	Ghi cache đồng thời ghi xuống RAM
No-write-allocate	Write miss ghi thẳng xuống RAM, không nạp vào cache
Read miss refill	Read miss đọc RAM rồi refill cache

FSM cache nên được mô tả theo các trạng thái: `IDLE`, `LOOKUP`, `HIT_READ`, `HIT_WRITE`, `MISS_READ_REQ`, `MISS_READ_WAIT`, `REFILL`, `MISS_WRITE_REQ`, `MISS_WRITE_WAIT` và `DONE`.

2.1.10. Main Memory có Delay

Người viết: Nguyễn Vũ Hoàng.

Main memory được mô phỏng như một RAM có độ trễ 5–10 chu kỳ. Tín hiệu `mem_ready` chỉ được kích hoạt khi dữ liệu thực sự sẵn sàng. Mục tiêu là mô phỏng gần hơn với thực tế của bộ nhớ ngoài, từ đó buộc CPU và cache phải stall đúng cách khi RAM chưa phản hồi.

2.1.11. Hazard trong Pipeline

Người viết: Nguyễn Nhật Phong.

Structural Hazard Structural hazard xuất hiện khi IF và MEM cùng cần truy cập unified cache. Cách xử lý là dùng `memory_arbiter`, ưu tiên MEM stage, còn IF stage sẽ bị stall nếu MEM đang dùng cache.

Data Hazard Ví dụ:

```
ADD R1, R2, R3
ADD R4, R1, R5
```

Cách xử lý là forwarding từ EX/MEM về EX và từ MEM/WB về EX, đồng thời không forward nếu thanh ghi đích là R0.

Load-use Hazard Ví dụ:

```
LW R1, 0(R0)
ADD R2, R1, R1
```

Cách xử lý là stall PC và IF/ID, đồng thời flush ID/EX để chèn bubble.

Control Hazard Control hazard xuất hiện với branch và jump. Cách xử lý là resolve branch tại EX stage; nếu branch taken hoặc jump thì flush các instruction sai đường đi.

2.2. Phân tích yêu cầu hệ thống

Người viết chính: Nguyễn Nhật Phong, bổ sung bởi cả nhóm.

2.2.1. Yêu cầu chức năng

Table 14: Yêu cầu chức năng của hệ thống

Nhóm chức năng	Yêu cầu
CPU Core	Thực hiện pipeline 5 tầng
ISA	Hỗ trợ R-type, I-type, J-type
Register File	8 thanh ghi, R0 = 0
ALU	ADD, SUB, AND, OR, SLT...
Memory	Load/store qua cache
Cache	Hit/miss, refill, write-through
Hazard	Stall, forwarding, flush
Tooling	Assembler <code>.asm</code> sang <code>.mem</code>
Verification	Testbench module và tích hợp

2.2.2. Yêu cầu phi chức năng

Các yêu cầu phi chức năng bao gồm: code Verilog rõ ràng và chia module hợp lý; interface giữa các module thống nhất; có testbench kiểm chứng từng phần; có waveform minh họa hoạt động; có tài liệu ISA, module interface, cache FSM; và có bảng kết quả pass/fail cho từng nhóm kiểm thử.

2.3. Thiết kế hệ thống

Người viết chính: Lê Hoàng Dũng

2.3.1. Tổng quan kiến trúc hệ thống

Người viết: Lê Hoàng Dũng.

Hệ thống được tổ chức theo top-level đã khóa gồm bốn khối chính nối tiếp nhau: `cpu_core`, `memory_arbiter`, `direct_mapped_cache` và `main_memory`. Cấu trúc này phản ánh kiến trúc

Von Neumann với một đường bộ nhớ thống nhất, trong đó CPU có hai cổng logic riêng cho nạp lệnh và truy cập dữ liệu, còn arbiter hợp nhất hai luồng này xuống một cổng cache vật lý.

`cpu_core` → `memory_arbiter` → `direct_mapped_cache` → `main_memory`

Về trạng thái hiện tại, dự án đã hoàn thiện pipeline 5 tầng gồm IF, ID, EX, MEM và WB. Các module hỗ trợ như `pipeline_regs`, `forwarding_unit` và `hazard_detection_unit` được tích hợp để xử lý data hazard, load-use hazard và control hazard. Khối `direct_mapped_cache` đã hoàn thiện FSM xử lý cache hit/miss, refill, write-through và phối hợp với RAM có delay thông qua `memory_arbiter`.

Table 15: Tổng quan các khối top-level

Khối	Vai trò chính
<code>cpu_core</code>	Thực thi pipeline 5 tầng, phát yêu cầu IF/MEM và nhận dữ liệu trả về từ hệ thống bộ nhớ
<code>memory_arbiter</code>	Điều phối hai cổng logic IF và MEM thành một cổng cache thống nhất, ưu tiên MEM khi tranh chấp
<code>direct_mapped_cache</code>	Cache trực tiếp ánh xạ, 16 lines x 1 word, hỗ trợ hit/miss, refill, write-through và no-write-allocate
<code>main_memory</code>	Bộ nhớ chính word-addressed, mô phỏng độ trễ và chỉ xử lý tối đa một request outstanding

2.3.2. Luồng pipeline và xử lý xung đột

Người viết: Lê Hoàng Dũng.

Pipeline được chia thành năm tầng. IF giữ PC và gửi yêu cầu đọc instruction; ID giải mã lệnh, đọc register file và tạo immediate; EX thực thi ALU, tính địa chỉ và quyết định branch; MEM phát yêu cầu load/store; WB ghi kết quả về register file. Các pipeline register IF/ID, ID/EX, EX/MEM và MEM/WB cần hỗ trợ ba cơ chế điều khiển quan trọng: **stall**, **flush** và **valid**.

Table 16: Quy tắc điều khiển pipeline và bộ nhớ

Điều kiện	Hành động
Cache miss	Stall toàn cục cho đến khi cache bật ready
Load-use hazard	Giữ PC và IF/ID, flush ID/EX để chèn bubble
Branch/jump taken	Flush IF/ID và ID/EX, đồng thời cập nhật PC tới target

Continued on next page

Table 16: Quy tắc điều khiển pipeline và bộ nhớ (Continued)

Điều kiện	Hành động
IF và MEM cùng truy cập cache	<code>memory_arbiter</code> ưu tiên MEM, IF bị stall trong chu kỳ tranh chấp
Ghi vào R0	Bỏ qua thao tác ghi, bảo toàn $R0 = 0$

2.3.3. Tổng quan giao diện module

Người viết: Lê Hoàng Dũng.

Các module dùng quy ước đặt tên tín hiệu thống nhất gồm `clk`, `rst`, `req`, `we`, `addr`, `wdata`, `rdata`, `ready`, `stall`, `flush`, `valid`, `hit` và `miss`. Ở top-level, `cpu_cache_top` nhận `clk`, `rst` và xuất các tín hiệu quan sát như `halted`, `debug_pc`. Bên trong, `cpu_core` tách hai cổng logic IF/MEM để mô tả đúng nhu cầu pipeline, còn `memory_arbiter` chuyển chúng thành request cache thống nhất.

Table 17: Các nhóm giao diện chính của hệ thống

Module	Nhóm cổng	Mô tả
<code>cpu_core</code>	IF logical port	<code>if_req</code> , <code>if_addr</code> , <code>if_rdata</code> , <code>if_ready</code> dùng để nạp instruction
<code>cpu_core</code>	MEM logical port	<code>mem_req</code> , <code>mem_we</code> , <code>mem_addr</code> , <code>mem_wdata</code> , <code>mem_rdata</code> , <code>mem_ready</code> dùng cho load/store
<code>memory_arbiter</code>	Cache port	<code>cache_req</code> , <code>cache_we</code> , <code>cache_addr</code> , <code>cache_wdata</code> , <code>cache_rdata</code> , <code>cache_ready</code>
<code>direct_mapped_cache</code>	CPU/arbiter side	<code>req</code> , <code>we</code> , <code>addr</code> , <code>wdata</code> , <code>rdata</code> , <code>ready</code> , <code>hit</code> , <code>miss</code>
<code>direct_mapped_cache</code>	RAM side	<code>mem_req</code> , <code>mem_we</code> , <code>mem_addr</code> , <code>mem_wdata</code> , <code>mem_rdata</code> , <code>mem_ready</code>
<code>main_memory</code>	Memory port	<code>req</code> , <code>we</code> , <code>addr</code> , <code>wdata</code> , <code>rdata</code> , <code>ready</code> với mô hình một request outstanding

2.3.4. Ghi chú phạm vi thiết kế

Người viết: Lê Hoàng Dũng.

Phiên bản kiến trúc hiện tại tập trung vào CPU pipeline 5 tầng đơn giản, cache blocking và hệ thống bộ nhớ một cấp. Các kỹ thuật nâng cao như superscalar, out-of-order execution, branch prediction, multi-level cache, write-back cache, non-blocking cache và nhiều request bộ nhớ outstanding được xem là ngoài phạm vi để giữ thiết kế phù hợp với mục tiêu mô phỏng.

học thuật.

2.3.5. Kiến trúc tổng thể

Sơ đồ tổng thể của hệ thống có thể mô tả theo cấu trúc sau:

```

CPU Core 5-stage pipeline
IF -> ID -> EX -> MEM -> WB
|
| IF request + MEM request
v
Memory Arbiter
|
v
Unified Direct-Mapped Cache
|
v
Main Memory co delay

```

Table 18: Vai trò các khối chính

Khối	Vai trò
cpu_core	Thực hiện pipeline CPU
memory_arbiter	Điều phối IF/MEM request
direct_mapped_cache	Cache dùng chung instruction/data
main_memory	RAM có delay
cpu_cache_top	Kết nối toàn bộ hệ thống

2.3.6. Thiết kế module CPU Core

Người viết: Nguyễn Nhật Phong.

Các module cần mô tả gồm: pc_unit, instruction_decoder, control_unit, register_file, immediate_generator, alu, forwarding_unit, hazard_detection_unit, cùng các pipeline register: pipeline_reg_if_id, pipeline_reg_id_ex, pipeline_reg_ex_mem, pipeline_reg_mem_wb. Mỗi module cần được trình bày rõ input/output và vai trò trong luồng

xử lý.

2.3.7. Thiết kế Datapath & ALU

Người viết: Vũ Xuân Hải.

Phần này cần trình bày sơ đồ datapath, vai trò của ALU, register file, immediate generator và các khối MUX. Ngoài ra, nên phân tích luồng dữ liệu cho từng loại lệnh: R-type, ADDI, LW, SW, BEQ/BNE và J.

2.3.8. Thiết kế Control Unit

Người viết: Nguyễn Nhật Phong.

Nội dung cần làm rõ cách giải mã `opcode/funct`, cách sinh control signal cho từng lệnh, cách xử lý `HALT`, và cơ chế xử lý branch/jump trong pipeline.

2.3.9. Thiết kế Pipeline Registers

Người viết: Nguyễn Nhật Phong.

Pipeline register có nhiệm vụ giữ dữ liệu trung gian giữa các stage. Cần mô tả dữ liệu lưu trong từng khối, cách reset về NOP, cũng như cách stall và flush khi gặp hazard.

Table 19: Thông tin lưu trong các pipeline register

Pipeline Register	Dữ liệu lưu
IF/ID	PC, instruction
ID/EX	PC, register data, immediate, control signals
EX/MEM	ALU result, write data, branch target, control signals
MEM/WB	Memory data, ALU result, write register, control signals

2.3.10. Thiết kế Hazard Detection & Forwarding

Người viết: Nguyễn Nhật Phong.

Công thức phát hiện load-use hazard:

```
ex_mem_read && ((ex_rt == id_rs) || (ex_rt == id_rt))
```


Cơ chế stall sử dụng các tín hiệu `pc_stall`, `if_id_stall` và `id_ex_flush`. Cơ chế forwarding hỗ trợ hai hướng chính: $EX/MEM \rightarrow EX$ và $MEM/WB \rightarrow EX$.

Table 20: Mã forwarding

Giá trị	Ý nghĩa
2'b00	Dữ liệu gốc từ ID/EX
2'b01	Forward từ MEM/WB
2'b10	Forward từ EX/MEM

2.3.11. Thiết kế Memory Arbiter

Người viết: Nguyễn Vũ Hoàng.

Arbiter là khối điều phối truy cập cache giữa IF stage và MEM stage. Quy tắc ưu tiên là: nếu `mem_req = 1` thì phục vụ MEM stage; nếu không, nếu `if_req = 1` thì phục vụ IF stage. Khi MEM chiếm cache, IF phải stall.

```
IF request  ----\
> Memory Arbiter -> Cache
MEM request ----/
```

2.3.12. Thiết kế Direct-Mapped Cache

Người viết: Nguyễn Vũ Hoàng.

Phần này cần trình bày cấu trúc cache gồm `valid array`, `tag array`, `data array`, cùng cách xác định hit/miss và cơ chế xử lý với read hit, read miss, write hit, write miss.

Table 21: Các tình huống xử lý trong cache

Trường hợp	Hành động
Read hit	Trả dữ liệu từ cache
Read miss	Đọc RAM, refill cache

Continued on next page

Table 21: Các tình huống xử lý trong cache (Continued)

Trường hợp	Hành động
Write hit	Cập nhật cache và RAM
Write miss	Ghi thẳng RAM, không allocate

2.3.13. Thiết kế Main Memory

Người viết: Nguyễn Vũ Hoàng.

Main memory cần hỗ trợ kích thước mô phỏng phù hợp, cho phép nạp nội dung bằng `$readmemh`, cấu hình `MEM_DELAY`, và chỉ bật `mem_ready` khi hoàn thành một request.

2.3.14. Thiết kế Python Assembler

Người viết: Lê Hoàng Dũng.

Assembler Python là công cụ chuyển chương trình assembly của CPU 16-bit sang machine code dạng hexadecimal để nạp vào bộ nhớ mô phỏng bằng file `.mem`. Công cụ này đóng vai trò cầu nối giữa đặc tả ISA và quá trình verification: thay vì viết trực tiếp từng word mã máy, nhóm có thể viết chương trình thử nghiệm bằng mnemonic dễ đọc, sau đó assembler tự động mã hóa đúng định dạng R-type, I-type và J-type.

Luồng xử lý của assembler gồm hai pha chính. Pha thứ nhất đọc file `.asm`, loại bỏ dòng trống/comment, nhận diện label và xây dựng bảng địa chỉ label theo word address. Pha thứ hai duyệt lại từng instruction, phân tích mnemonic/toán hạng, kiểm tra register/immediate/address, mã hóa thành instruction 16-bit và ghi từng word hex 4 ký tự vào file `.mem`.

Table 22: Chức năng chính của Python assembler

Chức năng	Mô tả
Tiền xử lý input	Đọc file <code>.asm</code> , bỏ comment, chuẩn hóa khoảng trắng và tách label khỏi instruction
Quản lý label	Lưu tên label và địa chỉ word tương ứng để phục vụ branch/jump
Parse register	Nhận dạng R0–R7, kiểm tra register nằm trong 3 bit hợp lệ
Parse immediate	Hỗ trợ immediate 6 bit có dấu cho I-type và kiểm tra vượt phạm vi
Mã hóa R-type	Sinh opcode, rs, rt, rd, funct cho các lệnh ALU
Mã hóa I-type	Sinh opcode, rs, rt, imm6 cho ADDI, LW, SW, BEQ, BNE

Continued on next page

Table 22: Chức năng chính của Python assembler (Continued)

Chức năng	Mô tả
Mã hóa J-type	Sinh opcode và địa chỉ 12 bit cho J; sinh F000 cho HALT
Xuất file <code>.mem</code>	Ghi mỗi instruction thành một dòng hex 16-bit để dùng với <code>\$readmemh</code>

Assembler bám sát quy ước ISA đã chốt. Với R-type, opcode bằng 4'h0 và phép toán được xác định bằng trường `funct`. Với I-type, immediate được sign-extend khi thực thi trong CPU, nên assembler cần kiểm tra giá trị nằm trong miền biểu diễn của `imm6`. Với branch, offset được tính tương đối so với lệnh kế tiếp theo công thức `label_address - (branch_pc + 1)`. Với jump, địa chỉ label được mã hóa trực tiếp vào trường 12 bit.

Table 23: Các nhóm lệnh assembler hỗ trợ

Nhóm	Lệnh	Dạng assembly
R-type	ADD, SUB, AND, OR, XOR, SLT, SLL, SRL	OP rd, rs, rt
I-type arithmetic	ADDI	ADDI rt, rs, imm6
I-type memory	LW, SW	LW/SW rt, imm6(rs)
I-type branch	BEQ, BNE	BEQ/BNE rs, rt, label
J-type	J	J label
System	HALT	HALT

Ví dụ chuyển đổi chương trình arithmetic cơ bản:

Input:

```
ADDI R1, R0, 5
ADDI R2, R0, 10
ADD  R3, R1, R2
HALT
```

Output:

```
1045
108A
0280
F000
```

Các lỗi cần được assembler phát hiện sớm gồm mnemonic không hợp lệ, sai số lượng toán hạng,

register ngoài R0–R7, immediate vượt phạm vi, label chưa định nghĩa và địa chỉ jump vượt 12 bit. Việc kiểm tra lỗi ở tầng assembler giúp giảm thời gian debug waveform, vì file `.mem` đầu vào đã được đảm bảo đúng định dạng trước khi chạy simulation.

2.4. Quá trình xây dựng hệ thống

Người viết chính: Cả nhóm, mỗi người viết phần mình.

2.4.1. Quy trình triển khai

Người viết: Nguyễn Nhật Phong.

Quá trình triển khai nên được trình bày theo thứ tự: chốt ISA và opcode, thiết kế interface module, code các module đơn lẻ, viết testbench module, tích hợp CPU pipeline, tích hợp cache và RAM, thêm hazard detection và forwarding, thêm branch/jump/flush, viết assembler, chạy test tích hợp, chụp waveform và đánh giá.

2.4.2. Xây dựng Datapath, ALU, Register File

Người viết: Vũ Xuân Hải.

Các module đã thực hiện: `alu.v`, `register_file.v`, `immediate_generator.v`, `mux.v`. Nội dung nên trình bày cách ALU xử lý từng phép toán, cách register file đọc/ghi, cách bảo vệ R0, cách immediate generator sign-extend/zero-extend, và testbench tương ứng như `alu_tb.v`, `register_file_tb.v`.

Table 24: Kết quả kiểm thử Datapath và Register File

Module	Test	Kết quả
ALU	ADD/SUB/AND/OR/SLT	Pass
ALU	zero/negative/overflow	Pass
Register File	Read/write	Pass
Register File	R0 luôn bằng 0	Pass

2.4.3. Xây dựng Cache, RAM và Memory Arbiter

Người viết: Nguyễn Vũ Hoàng.

Các module đã thực hiện: `direct_mapped_cache.v`, `cache_controller.v`, `main_memory.v`, `memory_arbiter.v`. Cần mô tả FSM cache, RAM delay, và cơ chế arbiter ưu tiên MEM hơn IF.

Table 25: Kết quả kiểm thử Cache, RAM và Arbiter

Module	Test	Kết quả
Main Memory	Delay và mem_ready	Pass
Cache	Read hit	Pass
Cache	Read miss + refill	Pass
Cache	Write hit	Pass
Cache	Write miss	Pass
Arbiter	MEM ưu tiên IF	Pass

2.4.4. Xây dựng Control, Pipeline và CPU Core

Người viết: Nguyễn Nhật Phong.

Các module đã thực hiện: `control_unit.v`, `instruction_decoder.v`, `pc_unit.v`, `pipeline_regs.v`, `hazard_detection_unit.v`, `forwarding_unit.v`, `cpu_core.v`, `cpu_cache_top.v`. Phần này cần mô tả cách PC hoạt động, cách decode instruction, cách sinh control signal, cách pipeline register truyền dữ liệu, cơ chế stall/flush/forwarding và cách tích hợp với cache.

Table 26: Kết quả kiểm thử CPU Core và pipeline

Chức năng	Kết quả
Fetch instruction	Pass
Decode opcode/funct	Pass
Execute ALU instruction	Pass
Write-back register	Pass
Load/store	Pass
Stall cache miss	Pass
Load-use hazard	Pass
Branch/jump flush	Pass

2.4.5. Xây dựng Testbench, Assembler và Demo Program

Người viết: Lê Hoàng Dũng.

Phần verification được xây dựng quanh công cụ `tools/assembler.py`. Quy trình làm việc đề xuất là viết chương trình kiểm thử trong thư mục `asm/`, chạy assembler để sinh file `mem/`, sau đó nạp file `.mem` vào testbench tích hợp. Cách tổ chức này giúp các test arithmetic, load/store, cache, hazard và branch có thể được viết bằng assembly dễ đọc, đồng thời vẫn tạo ra input đúng định dạng cho `main_memory`.

```
python tools/assembler.py asm/program_05_full.asm mem/program_05_full.mem
```

Sau khi sinh file `.mem`, nhóm chạy simulation tích hợp để quan sát pipeline, register file, cache hit/miss, stall/flush và kết quả cuối cùng:

```
iverilog -o build/cpu_cache_tb.vvp \
rtl/cpu/*.v \
rtl/memory/*.v \
rtl/top/*.v \
tb/cpu_cache_tb.v
vvp build/cpu_cache_tb.vvp
```

Các module/file đã thực hiện hoặc cần duy trì đồng bộ gồm `tb/cpu_cache_tb.v`, `tb/alu_tb.v`, `tb/register_file_tb.v`, `tb/cache_tb.v`, `tools/assembler.py`, `asm/*.asm`, `mem/*.mem`, `sim/run.do` hoặc `Makefile`. Khi ISA thay đổi opcode, funct hoặc format lệnh, bảng mã trong assembler cần được cập nhật cùng lúc với `control_unit`, `instruction_decoder` và tài liệu ISA để tránh sai lệch giữa phần mềm tạo mã và phần cứng thực thi.

Table 27: Các demo program đề xuất

Demo	Mục tiêu	Kết quả kỳ vọng
Arithmetic	Test ADD/ADDI	R3 = 15
Load/store	Test SW/LW	Memory và register đúng

Continued on next page

Table 27: Các demo program đề xuất (Continued)

Demo	Mục tiêu	Kết quả kỳ vọng
Cache	Test miss/hit	Lần đầu miss, lần sau hit
Hazard	Test load-use	Không sai dữ liệu
Branch	Test BEQ/flush	R3 = 9

3. Kết quả và bàn luận

Phần này trình bày kết quả kiểm thử, waveform đề xuất và phân tích mức độ hoàn thành yêu cầu để chứng minh khả năng làm chủ phương pháp thiết kế, mô phỏng và kiểm chứng phần cứng số hiện đại. **Người viết chính:** Lê Hoàng Dũng, phối hợp cả nhóm.

3.1. Chiến lược kiểm thử

Chiến lược kiểm thử gồm hai cấp độ: kiểm thử module đơn lẻ và kiểm thử tích hợp. Ở mức module, các khối như ALU, register file, control unit, main memory, cache và arbiter được kiểm tra độc lập. Ở mức tích hợp, hệ thống CPU core + arbiter + cache + RAM được mô phỏng cùng các chương trình assembly hoàn chỉnh.

3.2. Testbench từng module

Table 28: Danh sách testbench theo module

Testbench	Mục tiêu
alu_tb.v	Test phép toán và flag
register_file_tb.v	Test read/write, reset, R0
control_unit_tb.v	Test control signal
main_memory_tb.v	Test delay và mem_ready
cache_tb.v	Test hit/miss, refill, write-through
memory_arbiter_tb.v	Test ưu tiên MEM hơn IF
cpu_core_tb.v	Test pipeline core
cpu_cache_tb.v	Test tích hợp toàn hệ thống

3.3. Kiểm thử chương trình Assembly

Test 1: Arithmetic

```

ADDI R1, R0, 5
ADDI R2, R0, 10
ADD  R3, R1, R2
HALT

```

Kết quả kỳ vọng: $R1 = 5$, $R2 = 10$, $R3 = 15$.

Test 2: Load/Store

```

ADDI R1, R0, 20
SW   R1, 0(R0)
LW   R2, 0(R0)
HALT

```

Kết quả kỳ vọng: $\text{Memory}[0] = 20$, $R2 = 20$.

Test 3: Cache Hit/Miss

```

LW R1, 10(R0)
LW R2, 10(R0)
HALT

```

Kết quả kỳ vọng: lần LW đầu tiên là cache miss, lần thứ hai là cache hit.

Test 4: Load-use Hazard

```

LW  R1, 0(R0)
ADD R2, R1, R1
HALT

```

Kết quả kỳ vọng: CPU stall đúng và $R2 = R1 + R1$.

Test 5: Branch

```

ADDI R1, R0, 5

```



```

ADDI R2, R0, 5
BEQ  R1, R2, TARGET
ADDI R3, R0, 1
TARGET:
ADDI R3, R0, 9
HALT

```

Kết quả kỳ vọng: R3 = 9, instruction sai sau branch bị flush.

3.4. Waveform minh họa

Waveform nên thể hiện rõ các nhóm tín hiệu quan trọng theo bảng sau:

Table 29: Các tín hiệu cần minh họa trên waveform

Nhóm	Tín hiệu cần minh họa
Pipeline	pc, if_id_instr, id_ex_opcode, ex_mem_alu_result, mem_wb_data
Register	reg_write, write_reg, write_data
Hazard	stall, flush, forward_a, forward_b
Memory	mem_req, mem_we, mem_addr, mem_ready
Cache	cache_hit, cache_miss, cache_state, valid, tag, index
RAM	ram_req, ram_ready, delay_counter

Mỗi waveform nên có tên test, tập tín hiệu quan trọng, phần giải thích 3–5 dòng và kết luận pass/fail.

3.5. Bảng tổng hợp kết quả kiểm thử

Table 30: Bảng tổng hợp pass/fail

STT	Test case	Mục tiêu	Kết quả kỳ vọng	Kết quả thực tế	Đánh giá
1	ALU ADD	Kiểm tra cộng	Đúng kết quả		Pass
2	Register R0	R0 không đổi	R0 = 0		Pass
3	Cache miss	RAM refill cache	Miss rồi refill		Pass

Continued on next page

Table 30: Bảng tổng hợp pass/fail (Continued)

STT	Test case	Mục tiêu	Kết quả kỳ vọng	Kết quả thực tế	Đánh giá
4	Load-use hazard	Stall pipeline	Không sai dữ liệu		Pass
5	Branch flush	Xóa lệnh sai	R3 = 9		Pass

3.6. Đánh giá hệ thống

Người viết chính: Cả nhóm, tổng hợp bởi Nguyễn Nhật Phong.

3.6.1. Kết quả đạt được

- Đã xây dựng CPU 16-bit.
- Đã có 8 thanh ghi, R0 = 0.
- Đã có pipeline 5 tầng.
- Đã hỗ trợ các lệnh cơ bản.
- Đã có unified direct-mapped cache.
- Đã mô phỏng cache hit/miss.
- Đã mô phỏng RAM delay.
- Đã xử lý stall khi cache miss.
- Đã xử lý load-use hazard.
- Đã có forwarding cơ bản.
- Đã có flush khi branch/jump.
- Đã có assembler Python.
- Đã có testbench và waveform.

3.6.2. Hạn chế

Một số hạn chế hiện tại có thể bao gồm: ISA còn đơn giản, cache line size mới là 1 word, chưa có branch prediction, chưa hỗ trợ interrupt/exception, chưa tối ưu hiệu năng pipeline, chưa triển khai lên FPGA, chưa có cache nhiều cấp, và chưa có cơ chế write-back cache.

3.6.3. Hướng phát triển

Các hướng phát triển đề xuất gồm: mở rộng ISA, thêm lệnh shift/logic/compare đầy đủ hơn, tăng số lượng register, tăng cache line size, thêm write-back cache, branch prediction, thống kê instruction/data, triển khai trên FPGA và xây dựng trình mô phỏng trực quan cho pipeline.

3.7. Kết luận

Người viết: Cả nhóm, tổng hợp bởi Lê Hoàng Dũng.

Thông qua đề tài, nhóm đã thiết kế và mô phỏng thành công một CPU 16-bit theo kiến trúc Von Neumann với pipeline 5 tầng và unified direct-mapped cache. Hệ thống có khả năng thực thi các lệnh cơ bản, xử lý load/store, branch/jump, cache hit/miss, stall, forwarding và flush. Ngoài ra, nhóm cũng xây dựng assembler Python và hệ thống testbench để kiểm chứng từng module cũng như toàn bộ hệ thống. Kết quả đạt được giúp nhóm hiểu rõ hơn về cách CPU thực thi lệnh, cách pipeline hoạt động và cách bộ nhớ cache ảnh hưởng đến hiệu năng hệ thống.

4. Tài liệu tham khảo

References

- [1] American Psychological Association, “Paper format,” *APA Style*. [Online]. Available: <https://apastyle.apa.org/style-grammar-guidelines/paper-format>. Accessed: May 13, 2026.
- [2] University of Pittsburgh Library System, “IEEE Style,” *Citation Help*. [Online]. Available: <https://pitt.libguides.com/citationhelp/ieee>. Accessed: May 13, 2026.
- [3] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*. Cambridge, MA, USA: Morgan Kaufmann, 2019.
- [4] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition*. Cambridge, MA, USA: Morgan Kaufmann, 2020.
- [5] J. von Neumann, *First Draft of a Report on the EDVAC*. Moore School of Electrical Engineering, University of Pennsylvania, 1945. [Online]. Available: <https://library.si.edu/digital-library/book/firstdraftofrepo00vonn>. Accessed: May 15, 2026.

- [6] IEEE, *IEEE Standard for Verilog Hardware Description Language*, IEEE Std. 1364-2005, 2006. [Online]. Available: <https://standards.ieee.org/ieee/1364/3641>. Accessed: May 15, 2026.

5. Phụ lục

Người phụ trách chính: Lê Hoàng Dũng.

5.1. Cấu trúc thư mục project

```
cpu_cache_project/
|
+-- rtl/
+-- tb/
+-- asm/
+-- mem/
+-- tools/
+-- sim/
+-- docs/
+-- README.md
```

5.2. Lệnh mô phỏng

```
iverilog -o build/alu_tb.vvp rtl/cpu/alu.v tb/alu_tb.v
vvp build/alu_tb.vvp
gtkwave wave/alu_tb.vcd
```

Test tích hợp:

```
python tools/assembler.py asm/program_05_full.asm mem/program_05_full.mem
iverilog -o build/cpu_cache_tb.vvp \
rtl/cpu/*.v \
```

```

rtl/memory/*.v \
rtl/top/*.v \
tb/cpu_cache_tb.v
vvp build/cpu_cache_tb.vvp
gtkwave wave/cpu_cache_tb.vcd

```

5.3. Code quan trọng

Nên đưa một số đoạn code đại diện cho ALU, control unit, cache FSM, hazard detection và assembler encode instruction; không nên đưa toàn bộ mã nguồn quá dài vào thân báo cáo.

```

1  module adder_4bit (
2      input  wire [3:0] a,
3      input  wire [3:0] b,
4      output wire [4:0] sum
5  );
6      assign sum = a + b;
7  endmodule
8

```

Listing 1: Example: 4-bit Verilog adder

5.4. Gợi ý chia file trong thư mục docs/

```

docs/
|
+-- 01_INTRODUCTION.md
+-- 02_THEORY_CPU_PIPELINE.md
+-- 03_ISA.md
+-- 04_DATAPATH_ALU.md
+-- 05_CONTROL_PIPELINE_HAZARD.md
+-- 06_CACHE_MEMORY.md
+-- 07_ASSEMBLER_TOOLING.md
+-- 08_VERIFICATION_TESTING.md
+-- 09_RESULT_EVALUATION.md
+-- 10_FINAL_REPORT.md

```

Table 31: Phân công file docs

File docs	Người viết chính	Nội dung
01_INTRODUCTION.md	Nguyễn Nhật Phong	Giới thiệu, mục tiêu, phạm vi
02_THEORY_CPU_PIPELINE.md	Nguyễn Nhật Phong	Von Neumann, pipeline, hazard
03_ISA.md	Lê Hoàng Dũng	ISA, opcode, format lệnh, assembler mapping
04_DATAPATH_ALU.md	Vũ Xuân Hải	Datapath, ALU, register file, immediate, MUX
05_CONTROL_PIPELINE_HAZARD.md	Nguyễn Nhật Phong	Control unit, PC, pipeline registers, forwarding, stall, flush
06_CACHE_MEMORY.md	Nguyễn Vũ Hoàng	Cache, RAM delay, memory arbiter, FSM
07_ASSEMBLER_TOOLING.md	Lê Hoàng Dũng	Assembler, file .asm, .mem, script mô phỏng
08_VERIFICATION_TESTING.md	Lê Hoàng Dũng	Testbench, demo program, waveform, pass/fail
09_RESULT_EVALUATION.md	Cả nhóm, Phong tổng hợp	Kết quả, hạn chế, hướng phát triển
10_FINAL_REPORT.md	Cả nhóm	Ghép báo cáo cuối

5.5. Checklist hoàn thành báo cáo

Checklist dưới đây chia nhỏ từng phần để nhóm dễ theo dõi tiến độ, đánh dấu hoàn thành và phân công phần còn thiếu trước khi nộp bản cuối.

Checklist chi tiết

- ☐ **Trang đầu.** Cập nhật tên đề tài, tên môn học, giảng viên hướng dẫn và thời gian — *Cả nhóm.*
- ☐ **Trang đầu.** Điền đầy đủ tên 4 thành viên nhóm — *Cả nhóm.*
- ☐ **Đóng góp.** Rà lại vai trò từng thành viên theo đúng phần đã thực hiện — *Cả nhóm.*
- ☐ **Đóng góp.** Bổ sung mức độ đóng góp hoặc ghi chú nếu giảng viên yêu cầu — *Cả nhóm.*
- ☒ **Giới thiệu.** Hoàn thiện bối cảnh CPU pipeline và cache — *Lê Hoàng Dũng.*

- ☒ **Giới thiệu.** Hoàn thiện mục tiêu đề tài theo yêu cầu project — *Lê Hoàng Dũng.*
- ☒ **Giới thiệu.** Rà phạm vi thực hiện và các giới hạn không triển khai FPGA — *Lê Hoàng Dũng.*
- ☒ **Lý thuyết.** Viết rõ kiến trúc Von Neumann và structural hazard — *Lê Hoàng Dũng.*
- ☐ **Lý thuyết.** Hoàn thiện mô tả CPU 16-bit, register file và nhóm lệnh — *Nguyễn Nhật Phong, Vũ Xuân Hải.*
- ☐ **Lý thuyết.** Hoàn thiện mô tả pipeline 5 tầng IF, ID, EX, MEM, WB — *Nguyễn Nhật Phong.*
- ☒ **ISA.** Chốt định dạng R-type, I-type và J-type — *Lê Hoàng Dũng.*
- ☒ **ISA.** Chốt bảng opcode và bảng funct — *Lê Hoàng Dũng.*
- ☒ **ISA.** Chốt quy ước assembly, branch, jump và HALT — *Lê Hoàng Dũng.*
- ☒ **ISA.** Chốt cờ ALU và ví dụ mã hóa lệnh — *Lê Hoàng Dũng.*
- ☐ **Datapath.** Viết chi tiết luồng dữ liệu qua từng stage — *Vũ Xuân Hải.*
- ☐ **Datapath.** Bổ sung mô tả ALU, phép toán và cờ trạng thái — *Vũ Xuân Hải.*
- ☐ **Datapath.** Bổ sung register file, luật R0 và cơ chế đọc/ghi — *Vũ Xuân Hải.*
- ☐ **Datapath.** Rà immediate generator và các MUX trong datapath — *Vũ Xuân Hải.*
- ☐ **Control.** Hoàn thiện control unit và bảng control signal — *Nguyễn Nhật Phong.*
- ☐ **Control.** Hoàn thiện pipeline registers, stall, flush và bubble — *Nguyễn Nhật Phong.*
- ☐ **Control.** Hoàn thiện hazard detection cho load-use hazard — *Nguyễn Nhật Phong.*
- ☐ **Control.** Hoàn thiện forwarding unit và mã chọn forwarding — *Nguyễn Nhật Phong.*
- ☐ **Control.** Hoàn thiện xử lý branch/jump flush — *Nguyễn Nhật Phong.*
- ☐ **Memory.** Viết rõ memory arbiter và quy tắc MEM ưu tiên IF — *Nguyễn Vũ Hoàng.*
- ☐ **Memory.** Hoàn thiện direct-mapped cache, tag/index/valid/data array — *Nguyễn Vũ Hoàng.*
- ☐ **Memory.** Hoàn thiện chính sách write-through và no-write-allocate — *Nguyễn Vũ Hoàng.*
- ☐ **Memory.** Hoàn thiện main memory có delay và tín hiệu **ready** — *Nguyễn Vũ Hoàng.*

- ☐ **Interface.** Rà top-level `cpu_cache_top`, `halted`, `debug_pc` — *Lê Hoàng Dũng.*
- ☐ **Interface.** Rà IF logical port và MEM logical port của `cpu_core` — *Lê Hoàng Dũng.*
- ☐ **Interface.** Rà chuẩn tín hiệu `req/we/addr/wdata/rdata/ready` — *Lê Hoàng Dũng.*
- ☐ **Tooling.** Hoàn thiện mô tả assembler Python và luồng `.asm` sang `.mem` — *Lê Hoàng Dũng.*
- ☐ **Tooling.** Bổ sung ví dụ input/output assembler — *Lê Hoàng Dũng.*
- ☐ **Kiểm thử.** Liệt kê testbench module đơn lẻ — *Lê Hoàng Dũng.*
- ☐ **Kiểm thử.** Liệt kê test tích hợp CPU/cache/RAM — *Lê Hoàng Dũng.*
- ☐ **Kiểm thử.** Bổ sung demo program arithmetic, load/store, cache, hazard, branch — *Lê Hoàng Dũng.*
- ☐ **Kiểm thử.** Điền kết quả thực tế trong bảng pass/fail — *Lê Hoàng Dũng.*
- ☐ **Kiểm thử.** Chèn hoặc mô tả waveform minh họa các tín hiệu chính — *Lê Hoàng Dũng.*
- ☐ **Đánh giá.** Hoàn thiện kết quả đạt được — *Cả nhóm.*
- ☐ **Đánh giá.** Hoàn thiện hạn chế của thiết kế — *Cả nhóm.*
- ☐ **Đánh giá.** Hoàn thiện hướng phát triển — *Cả nhóm.*
- ☐ **Kết luận.** Rà kết luận bám sát mục tiêu đề tài — *Cả nhóm.*
- ☐ **Tài liệu.** Rà citation, bibliography và định dạng link — *Cả nhóm.*
- ☐ **Phụ lục.** Rà kế hoạch làm việc, cấu trúc thư mục và lệnh mô phỏng — *Lê Hoàng Dũng.*
- ☐ **Final check.** Biên dịch PDF không lỗi LaTeX — *Cả nhóm.*
- ☐ **Final check.** Rà mục lục, số bảng, caption và bố cục bảng — *Cả nhóm.*

5.6. Lưu ý khi viết báo cáo

Nên viết báo cáo theo hướng mô tả quá trình phát triển hệ thống, không chỉ liệt kê module. Với mỗi phần, nên trả lời đủ bốn ý: mục đích của module/phần đó là gì, thiết kế hoạt động như thế nào, đã triển khai bằng file/module nào, và đã kiểm thử ra sao cùng kết quả thế nào.

Ví dụ với cache: mục đích là giảm độ trễ truy cập bộ nhớ chính; thiết kế sử dụng unified direct-mapped cache gồm `valid array`, `tag array` và `data array`; triển khai bằng các module như

`direct_mapped_cache.v`, `main_memory.v`, `memory_arbiter.v`; và kiểm thử bằng `cache_tb.v` với các trường hợp read hit, read miss, write hit và write miss.